

Abstract—The system manages over **3,400 total active agents** (2500+ light orbs, 500+ articulated pedestrians, 400 fireflies) alongside **1,200 environmental assets**, sustaining **60 FPS** performance despite the high-density agent count and complexity.

1 GEOMETRY & SPATIAL ARCHITECTURE

1.1 Parametric Modelling & Animation (2a)

The environment utilizes Parametric Representations to achieve infinite resolution with minimal data.

1.1.0.1 Procedural Terrain:

The topography is defined by $y(x, z) = h_{base}(x, z) - h_{river}(d)$. The river's centre-line c_x uses a dual-frequency meander ($A_m = 55, \lambda_m = 140$):

$$c_x(z) = A_m \sin\left(\frac{z + s}{\lambda_m}\right) + 0.35A_m \sin\left(\frac{z - s}{0.55\lambda_m}\right) \quad (1)$$

Vertical displacement is governed by Cubic Hermite Interpolation (smoothstep [1]) to ensure C^1 continuity. This mathematical formulation ensures smooth surface normals and eliminates specular artifacts at boundaries.

1.1.0.2 Reactive Bézier Canopy:

A "Glowing Blanket" is implemented as a tensor-product cubic Bézier surface evaluated using Bernstein basis polynomials:

$$S(u, v) = \sum_{i=0}^3 \sum_{j=0}^3 B_i^3(u) B_j^3(v) P_{i,j}(t) \quad (2)$$

The 16 control points $P_{i,j}$ are animated via $P_{i,j} \cdot y = A \sin(\omega t + \phi_{i,j})$. By manipulating handles rather than static vertices, we achieve low-level deformable animation with $O(1)$ update complexity per frame, avoiding skeletal overhead.

1.2 Spatial Discretisation & Optimization (2b)

To resolve the $O(N^2)$ agent interaction bottleneck, we evaluated a Spatial Hash ($s = 10m$) against a Point Quadtree. While the Hash mapping $(i, j) = (\lfloor x/s \rfloor, \lfloor z/s \rfloor)$ reduced complexity to $O(N \cdot k)$, the Quadtree was selected for its adaptive subdivision of non-uniform agent densities. To avoid the "Anti-Pattern" of memory fragmentation, we utilized a pre-allocated 4096-node pool, eliminating per-frame heap allocation and Garbage Collection (GC) spikes. At $N = 1000$ (≈ 50 FPS), the Quadtree provided superior cache coherency and reduced frame-time variance during rapid camera shifts, ensuring stable Temporal Continuity for the simulation.



(a) Grid Overlay (b) Occupancy (c) Query Region

Fig. 1: Spatial optimisation systems.

2 CROWD SIMULATION (3a)

I implemented three distinct behavioral systems that orchestrate conflicting objectives: complex emergent behavior versus real-time scalability.

2.0.1 Orb Swarm: Procedural Fluidity and Stochastic Flow

The river corridor is populated by up to 6300 "Light Agents" (at 60FPS) utilizing a Structure-of-Arrays layout for cache efficiency. Their movement follows a stochastic flow model where forward motion along the Z-axis is coupled with a lateral parametric swerve. The target offset T_i is defined by:

$$T_i(z) = A_s \sin(z_i \cdot f_s + \phi_i). \quad (3)$$

To achieve smooth, non-jittery transitions, steering is resolved via a spring-damper model: $\dot{v}_{x,i} = (T_i - dx_i) \cdot K_s \cdot dt$, where K_s represents stiffness, moving away from rigid kinematic paths toward reactive force-driven movement.

2.0.2 Walkway Pedestrians: Finite State Machine and Prioritized Dithering

For the riverbank walkways, I implemented a Finite State Machine (FSM) to handle behaviour. Agents transition between states (CRUISE, QUEUE, AVOID, LEADER) based on local density and proximity to "Ghost Targets". To prevent "force cancellation", I utilized a Prioritised Dithering Accumulator. Steering forces are accumulated in order of importance (e.g., P_1 Separation $> P_3$ Flow-field). If the accumulated force exceeds $0.8 \cdot F_{max}$, lower-priority calculations are truncated to ensure safety-critical avoidance is never overridden.

2.0.3 Street Intelligence: Attraction and Environmental Awareness

The hilltop streets models "Context-Aware" intelligence. Agents are stochastically attracted to the Glowing Pyramid, modelled as a horizontal-only pull:

$$\vec{F}_{attr} = 0.08 \cdot w_a \cdot \hat{d}_{pyramid} \quad (4)$$

This focal point attraction is balanced against a sophisticated avoidance system that handles three distinct geometry types:

- **Pyramid Avoidance:** A high-priority radial repulsion zone (25 units).
- **Light Cube Avoidance:** Dynamic avoidance of atmospheric installations to simulate pedestrians navigating a crowded festival.
- **Procedural House Avoidance:** Rectangular AABB collision detection, ensuring agents remain on the carved streets.

This multi-layered approach ensures that while agents “Wander” randomly, they remain strictly constrained by the procedural environment.

3 PROCEDURAL KINEMATICS AND ARTICULATED ANIMATION (3b)

3.0.1 Hierarchical Skeleton and Orientation Dynamics

The humanoid agent’s orientation is resolved using Quaternions to ensure smooth, C^1 continuous facing changes, allowing for Spherical Linear Interpolation toward the velocity vector $\vec{v}$. The orientation q at time t is calculated as:

$$q_{mesh} = \text{Slerp}(q_{current}, q_{target}, \alpha) = \frac{\sin(t\Omega)}{\sin\Omega} q_{target} \quad (5)$$

where Ω is the angle between quaternions. This guarantees a constant angular velocity and unit magnitude, preventing “shrinking” artifacts associated with linear interpolation.

3.0.2 Velocity-Reactive Sinusoidal Walk Cycles

The gait is generated dynamically through a series of sinusoidal oscillations. To maintain physical realism, the walk frequency f scales linearly with the magnitude of the agent’s velocity $|\vec{v}|$:

$$f_{walk} = f_{base} + k|\vec{v}|, \quad \text{where } f_{base} = 2Hz \quad (6)$$

Leg and arm swings are phased π radians apart to maintain the center of mass. The angular displacement θ for a limb is defined by:

$$\theta(t) = A_{max} \cdot \sin(2\pi f_{walk}t + \phi) \quad (7)$$

A vertical “bob” (Δy) is added to the torso to simulate the human gait’s arc of suspension, with amplitude coupled to speed to distinguish between a “stroll” and a “brisk walk.”

4 RENDERING OPTIMISATION PIPELINE (3c)

4.0.1 Eliminating the Draw Call Bottleneck via Hardware Instancing

I implemented a Hardware-Accelerated Instancing pipeline using `THREE.InstancedMesh`, since rendering 1000+ individual agents using standard drawing cells would result $O(n)$ overhead. This system consolidates the entire swarm into a single draw call by sharing a vertex buffer and material across all instances. Each agent’s unique state (position, rotation, and scale) is stored in a 4×4 transformation matrix $\mathbf{M}_i$, which is passed to the GPU as an Instanced Attribute:

$$\mathbf{V}_{world} = \mathbf{M}_i \times \mathbf{V}_{local} \quad (8)$$

This architectural shift moves the complexity from the CPU’s command processor to the GPU’s fixed-function instancing hardware, enabling a sustained 60 FPS even with many agents.

4.0.2 Optimised CPU-to-GPU Data Flow and SoA Layout

To minimise bus traffic across the PCIe interface, I adopted a Data-Oriented Design using a Structure-of-Arrays layout for agent states. Instead of an “Array of Objects,” agent data is stored in contiguous `Float32Arrays`:

```
pos_z[N], offset_x[N], vel_x[N], phase[N].
```

This layout ensures CPU Cache Coherency during the per-frame matrix update loop. The transformation matrices are

written directly into the instanceMatrix buffer. To inform the driver of these frequent updates, I set the buffer usage to `THREE.DynamicDrawUsage`, which enables the driver to use optimal memory mapping strategies.

4.0.3 Spatial Chunking and Frustum Culling

For further scalability, I implemented a Spatial Chunking system. Agents are grouped into spatial regions (up to 512 per chunk), each represented by its own `InstancedMesh`. By setting `frustumCulled = true`, I leverage Three.js’s underlying spatial hierarchy to automatically cull chunks outside the camera’s view frustum. This ensures that matrix construction and GPU uploads are skipped for off-screen agents, reducing the per-frame bandwidth requirement from $O(N)$ to $O(N_{visible})$.

5 ADAPTIVE RENDERING & VISUAL SIGNALS

5.1 Adaptive Level of Detail (4a)

To sustain 6700+ entities at 60 FPS, we implemented a multi-tiered LOD strategy to balance geometric complexity.

- **Discrete Environment LOD:** Assets use three tiers: **LOD0** (Full/Shadows), **LOD1** (Simplified), and **LOD2** (Box silhouette, no shadows). This reduces vertex pressure by $> 70\%$ for background geometry.
- **Instanced Agent LOD & Hysteresis:** 1,000+ orbs swap to 8×8 “sparks” at distance. To prevent Temporal Aliasing, a hysteresis buffer ($H = 10$) ensures stable state transitions at boundaries.
- **Animation LOD:** Pedestrians are the primary CPU bottleneck. We implemented Kinematic Decimation: **Near** agents use full C^1 walk-cycles; **Mid** update every N frames; **Far** use cached neutral poses. This cut animation overhead by $\approx 60\%$.
- **Light/Shadow Pooling:** To bound Fragment Shader cost, we implemented a Light Pool. Only the 12 nearest lamps are assigned active `SpotLight` objects. PCF Shadows are culled beyond 25 units, significantly reducing VRAM bandwidth.

5.2 Signal Processing & Light Transport (4b)

To resolve **Spatial Aliasing** in high-frequency geometry (railings/roofs), we use a dual-pass approach. Hardware MSAA handles geometric sub-sampling, while a post-process FXAA pass detects Luma discontinuities to smooth shader-based edges and alpha-tested textures (the rocks) missed by the rasterizer.

To unify the “Lumiere” aesthetic, we implemented a **bloom pipeline** to simulate optical scattering:

- 1) **Bright-pass:** Isolates emissive signals ($l_{in} > 0.3$).
- 2) **Gaussian Convolution:** Models light spread ($r=0.4$).
- 3) **Additive Blending:** Recombines signals prior to Tone Mapping to preserve HDR intensities (up to 6.0).

This pipeline, combined with time-reactive HSL shifts in the Bézier canopy, ensures atmospheric coherence and high visual fidelity.

REFERENCES

- [1] Wikipedia contributors. Smoothstep, 2024.